AIDA vs AWS Kiro

AIDA positioning · best-effort comparison — live source: github.com/joemooney/aida

rendered 2026-07-10

Last updated: 2026-07-09

The TL;DR: **Kiro and AIDA both believe specs should be durable, structured artifacts — not throwaway prompts.** Kiro delivers that as a polished, AWS-backed agentic **IDE** with EARS-notation requirements and task→requirement traceability *inside* each feature. AIDA delivers it as a **vendor-neutral, git-canonical graph** that lives across your whole project and is readable by *any* agent via MCP. If you want a beautiful integrated editor that produces well-structured specs for the feature in front of you, Kiro is excellent. If you want the specs to be a cross-cutting graph that outlives any one feature and isn't tied to a single vendor's editor, that's AIDA's bet.

Kiro is, alongside Spec Kit, one of AIDA's two nearest competitors — and the one that most explicitly adopted the "specs as durable artifacts" language.

What Kiro is

Kiro is AWS's agentic IDE (standalone editor; preview 2025, GA May 2026, replaced Amazon Q; closed and usage-metered) built around **spec-driven development**. Its signature flow: from a prompt, Kiro generates a structured spec set per feature —

- `requirements.md` — requirements in **EARS notation** (Easy Approach to Requirements Syntax: *"WHEN <trigger>, the system SHALL <response>"*), which is genuinely rigorous structured-requirements discipline.
- `design.md` — the technical design derived from the requirements.
- `tasks.md` — an ordered task list, with **tasks that link back to the requirements they implement** (task→requirement traceability).

Plus **agent hooks** (event-triggered agent actions, e.g. "regenerate tests when this file changes") and the usual integrated-IDE agent affordances.

It is a strong, well-funded product. **Be precise about what it has** — Kiro genuinely advanced the SDD bar:

- **EARS-notation requirements** are *more* structured along the requirement-phrasing axis than AIDA's freeform descriptions.
- **Task→requirement traceability** exists — *within a feature's spec set.*

- **“Specs as artifacts”** is its explicit pitch — the vocabulary AIDA also uses. (This is why “AIDA has specs as artifacts” is no longer, by itself, a differentiator — it’s table-stakes vocabulary now.)

Where Kiro holds up (and you should just use it)

- **You want a polished, integrated editor experience.** Kiro is a full IDE; AIDA is a CLI + MCP layer that rides whatever editor/agent you already use. If the IDE *is* the product you want, Kiro delivers it.
- **EARS rigor matters to you.** If precise, testable requirement phrasing is a priority, Kiro’s EARS scaffolding is a real strength.
- **You’re an AWS shop.** First-party AWS backing and integration is a rational reason to choose it.
- **Single-feature, single-vendor scope is fine.** When work is one feature at a time inside one editor, Kiro’s per-feature spec set is a clean fit.

If that’s your shape, **use Kiro.**

Where Kiro starts to crack — and what AIDA adds

The cracks share two roots: Kiro’s specs are **per-feature artifact sets** (not a cross-cutting graph), and Kiro is **vendor- and editor-bound** (the specs live in, and are produced by, the Kiro IDE).

Symptom as the project / team grows	Kiro	AIDA
“Query the spec graph from Claude Code / Codex / Cursor”	Specs are produced and consumed inside the Kiro IDE	A token-efficient CLI (primary surface) reads the plain-git store on any agent; an MCP server exposes the graph as typed tools for MCP clients
“What’s blocked across this epic, across features?”	Traceability is task→requirement *within* a feature set	Typed cross-feature `BlockedBy`/`parent`/`child` graph, walkable across the whole project
“These three features share a requirement — rename it once”	Requirements are scoped to a feature’s `requirements.md`	Stable global IDs resolve to one UUID; one edit, every reference current
“Does the code still trace to its spec, enforced at commit?”	Task↔requirement links are authored in the spec set; no commit-gated code↔spec enforcement loop	`// trace:SPEC-ID` + commit-trailer linkage + lifecycle auto-bump on merge

Symptom as the project / team grows	Kiro	AIDA
“Run an orchestrated multi-agent drain with escalation”	Agent hooks are event-triggered IDE actions, not a cross-agent drain	Orchestrated multi-phase drain with spec-grounded escalation + shelving across a vendor-neutral fleet
“Keep the specs if we switch editors / agents”	Tied to the Kiro IDE	Plain git + open MCP — portable across every vendor by construction

The honest framing: **Kiro is the best integrated editor for producing a feature’s structured specs. AIDA is the vendor-neutral graph underneath that keeps every feature’s specs stable, related, traced, and queryable by any agent — independent of which editor produced them.**

The honest caveats (don’t let AIDA overclaim)

- **Kiro’s UX and funding dwarf AIDA’s.** A polished AWS-backed IDE is a different category of product investment. AIDA competes on the structured-graph + portability layer, not on editor experience.
- **EARS is a real edge for Kiro on requirement phrasing.** AIDA does not impose EARS as its canonical schema; if testable requirement syntax is your priority, Kiro’s notation-first authoring is a point for Kiro. AIDA’s answer is an **optional EARS lens**, not a mandate: `aida lint <SPEC|--scope feature|task|story>` runs a read-only, deterministic (no-LLM) heuristic pass that flags vague triggers, missing expected behavior, conflicting constraints, and low-testability wording, and prints suggested EARS-style rewrites as drafts. It never mutates the canonical spec — AIDA stays a graph-first substrate, EARS is layered on top as a quality lens you opt into.
- **“Specs as artifacts” is no longer differentiating.** Kiro adopting the vocabulary means AIDA must lead with the *graph + enforcement + portability*, not the slogan.
- **Composable in principle.** Nothing stops a team from drafting EARS specs in Kiro and holding the cross-feature graph + traces + lifecycle in AIDA — though the integration is less natural than the Spec-Kit-in-your-agent case, since Kiro is its own editor.

When to use which

Use **Kiro** when…	Use **AIDA** when…
You want a polished, integrated agentic IDE	You want to keep your editor/agent and add a spec graph under it

Use **Kiro** when…	Use **AIDA** when…
EARS-notation requirement rigor is a priority	Cross-cutting relationships + stable IDs + trace enforcement matter more
Single-feature, single-vendor scope is fine	Specs must be queryable by any agent, portable across vendors
AWS-first-party backing is decisive	Git-canonical, vendor-neutral system-of-record is decisive
The spec set is per-feature	The spec graph must outlive every individual feature and editor

---

Bottom line

Kiro pushed the SDD bar — EARS rigor, task→requirement traceability, "specs as artifacts" — and it's a genuinely strong, well-funded IDE. AIDA's claim is orthogonal to the editor: **the value isn't in producing one feature's specs beautifully, it's in keeping every feature's specs a stable, related, traced, queryable graph that no single vendor's editor owns.** Kiro proves teams want durable structured specs; AIDA's bet is that, at scale, they'll also want those specs to be a cross-cutting graph they can carry across every agent and editor — because it lives in git.

See also: `vs-spec-kit.md` (the other nearest competitor) and `docs/competitive-analysis/2026-05-31-round2-moat-gaps-moves.md` (the full competitive picture, including why "specs as artifacts" is now table-stakes vocabulary).